

Collections & Streams in C#

SWEN1 | BIF3

Table of Contents

- Collections in .Net
- Streams in .Net
- File I/O



Collections in .Net

Introduction

- Collections are containers for multiple objects
- Objects may be added, iterated, searched and removed (during runtime)
- Different implementations exist using different algorithms (belonging to the use case)

Generic container types:

- set .. arbitrary collection of objects (unsorted, unordered)
- list .. objects are in a sequence (have an order - but are not sorted)
- dictionary .. key/value pairs

Collection classes

see https://docs.microsoft.com/en-us/dotnet/standard/collections/

- Alternative to an array (sized) is a collection with often extended features.
- Implement **ICollection** (which inherits from **IEnumerable**)
  - IEnumerable enables possibility to iterate
- Further features
  - generic
  - immutable
  - concurrent
  - sorted
  - blocking

Generic Collections

System.Collections.Generic

Class	Description
List	Collection with ability to adapt size
Stack	LIFO Collection
Queue	FIFO Collection
Dictionary	Key-Value-Pair Collection
SortedList	Key-Value-Pair Collection (key is the sorting criteria)

Common Collections

- ***ArrayList<>*** Represents an array of objects whose size is dynamically increased as required. Usually NOT used due to performance and extension reasons.
- ***Hashtable<>*** Represents a collection of key/value pairs that are organized based on the hash code of the key.
- ***Queue<>*** Represents a first in, first out (FIFO) collection of objects.
- ***Stack<>*** Represents a last in, first out (LIFO) collection of objects.
- ***Dictionary<Key, Value>*** Represents a collection of key/value pairs that are organized based on the key.
- ***List<>*** Represents a list of objects that can be accessed by index. Provides methods to search, sort, and modify lists.
- ***SortedList<Key, Value>*** Represents a collection of key/value pairs that are sorted by key based on the associated ***IComparer*** implementation.

Thread Safe Collections

System.Collections.Concurrent

Class	Description
ConcurrentBag	Thread-safe, unordered collection of objects
ConcurrentStack	Thread-safe LIFO Collection
ConcurrentQueue	Thread-safe FIFO Collection
ConcurrentDictionary	Thread-safe Key-Value-Pair Collection

see https://learn.microsoft.com/de-de/dotnet/api/system.collections.concurrent?view=net-7.0

Thread Safe Collections

- ***BlockingCollection<>*** Provides bounding and blocking functionality for any type that implements ***IProducerConsumerCollection<>***
- ***ConcurrentQueue<>*** Thread safe implementation of a FIFO (first in, first out) queue.
- ***ConcurrentStack<>*** Thread safe implementation of a LIFO (last in, first out) stack.

In C# the ***ConcurrentQueue*** and ***ConcurrentStack*** classes do NOT (!!!) use locks at all. Instead, they rely on Interlocked operations to achieve thread safety.

- ***ConcurrentBag<>*** Thread safe implementation of an unordered collection of elements.
- ***ConcurrentDictionary<TKey,TValue>*** Thread safe implementation of a dictionary of key value pairs.
- ***IProducerConsumerCollection<>*** The interface that a type must implement to be used in a ***BlockingCollection***

Enumerator / Enumerable

- ***IEnumerator*** or ***IEnumerator<>***
  Supports a simple iteration over a collection.
- ***IEnumerable*** or ***IEnumerable<>***
  Exposes an enumerator, which supports a simple iteration over a non generic collection.

Cost (resources, performance) of using collections

- ***Initial capacity***: the number of elements a collection can hold before it needs to grow
  - memory consumption –vs– performance impact on resize
- ***Growth policy***: how a collection expands when ist capacity is exceeded
  - frequent resizing may cost performance

Collection	Initial Capacity (default)	Growth Policy (default	Cost of Append	Cost of Insert	Cost of Delete
List	0 elements	2x	O(1) ... O(n) on resize	O(n) due to shifting elements	O(n) due to shifting elements
Dictionary	0 elements	2x	O(1) ... O(n) on Hash collisions		O(1) element found O(n) on hash collisions
LinkedList	No, grows dynamically	dynamic	O(1) always	O(1) ... O(n)	O(1) node reference O(n) searching

Choosing the right collection

- memory-consumption
  - size . . . the amount of stored objects
  - capacity . . . the amount of reserved memory (for future objects)
- performance
  - fast access of items
  - fast search for items
  - fast modifications
  - fast sorting

Hands-On – Create a collection and add items

```
public struct TaskItem {
  public string Text;
  public DateTime Created;
  public bool Status;
}
```

```
// create collection and add some items
List<TaskItem> tasks = new()
{
  new TaskItem() { Text="Work on Self-Study", Created=DateTime.Now, Status=true },
  new TaskItem() { Text="Prepare for Presentation", Created=DateTime.Now },
};
tasks.Add(new TaskItem() { Text = "Learn fancy CS* syntax", Created = DateTime.Now } );

// fetch a specific item
Console.WriteLine($"My first task is {tasks[0].Text}");

// show all items
foreach (var task in tasks)
{
  Console.WriteLine(task.Text);
}
```

References

- C# Collections: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/collections
- C# Thread-safe Collections: https://docs.microsoft.com/en-us/dotnet/standard/collections/thread-safe/
- .Net Interlocked: https://docs.microsoft.com/en-us/dotnet/api/system.threading.interlocked

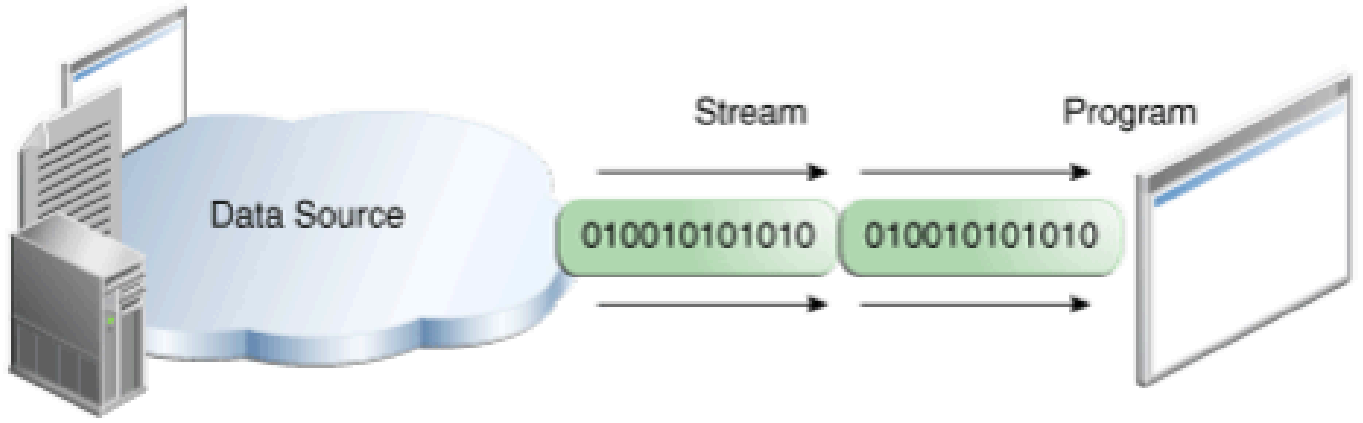


Streams in .Net

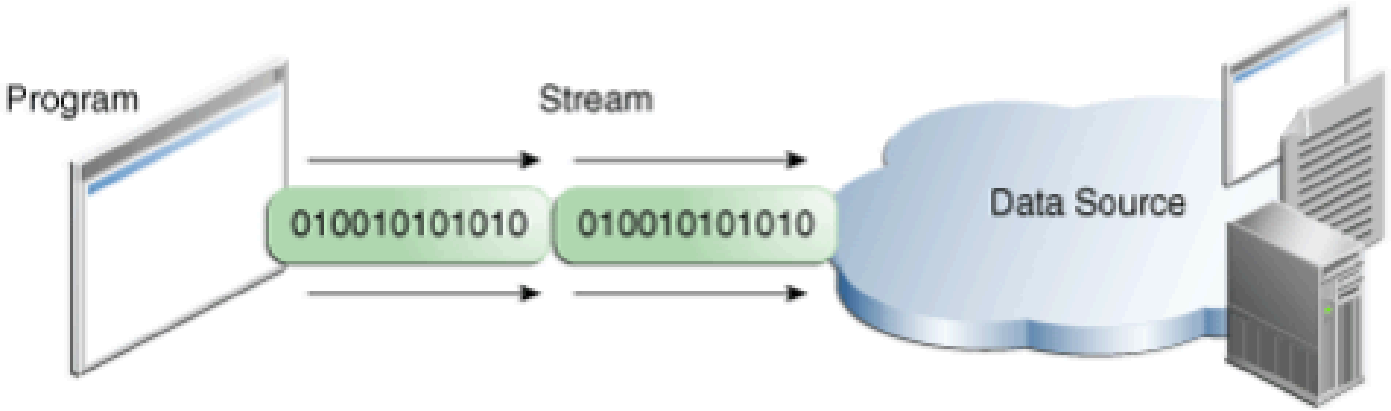
What is a stream?

- a sequence of data
- provide a common view of data sources
- isolate the programmer from the specific details of the underlying system(s)
- read, write and seek operations

See https://docs.microsoft.com/en-us/dotnet/standard/io/composing-streams



Reading information into a program.



Writing information from a program.

Classes for I/O via Streams

Input

Byte-Stream (bytes)	Text-Stream (chars)
Stream .Read(...) .CanRead() .Close()	TextReader .Read(...) .ReadLine() .Close()
BinaryReader .Read()	StringReader .ReadLine()

Output

Byte-Stream (bytes)	Text-Stream (chars)
Stream .Write(...) .Flush() .Close()	TextWriter .Write(...) .WriteLine() .Flush() .Close()
BinaryWriter	StringWriter

- ***TextWriter/Reader*** is an abstract class. It provides an abstraction for writing/reading character based data to/from a data source.
- ***StringWriter/Reader*** is a concrete implementation that uses a StringBuilder/string as data source.
- ***StreamWriter/Reader*** is a concrete implementation that uses a writeable/readable Stream as data source. Since a Stream is abstraction for writing/reading byte based data, an Encoding instance is required for the translation between characters and bytes.

Hands-On – Read/Write from/to a Text-File

- Write a Text-File:

```
// Write file using StreamWriter
using (StreamWriter writer = new StreamWriter(fullPath))
{
  writer.WriteLine("Monica Rathbun");
  writer.WriteLine("Vidya Agarwal");
  writer.WriteLine("Mahesh Chand");
  writer.WriteLine("Vijay Anand");
  writer.WriteLine("Jignesh Trivedi");
}
```

- Read a Text-File:

```
// Read file using StreamReader. Reads file line by line
using(StreamReader file = new StreamReader(textFile)) {
  int counter = 0;
  string In;

  while ((In = file.ReadLine()) != null) {
    Console.WriteLine(ln);
    counter++;
  }
  file.Close();
  Console.WriteLine($ "File has {counter} lines.");
}
```

```
// Read entire text file content in one string
string text = File.ReadAlliext(textFile);
Console.WriteLine(text);
```

LINQ Language-integrated Query

- query capabilities directly in the C# language
- type checking at compile time
- IntelliSense support in Visual Studio IDE
- consistent query experience for objects (LINQ to Objects), relational databases (LINQ to SQL), and XML (LINQ to XML)

see https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/

LINQ Cheat-Sheet

Filtering

```
var col = from o in Orders
          where o.CustomerID == 84
          select o;
```

```
var col2 = Orders.Where(o => o.CustomerID == 84);
```

Ordering

```
var col = from o in orders
          orderby o.Cost ascending
          select o;
```

```
var col2 = orders.OrderBy(o => o.Cost);
```

Paging (using Skip & Take)

```
//skip first 2 and return the 2 after

var col3 = (from o in orders
            where o.CustomerID == 84
            orderby o.Cost
            select o).Skip(2).Take(2);
```

```
var col3 = (from o in orders
            where o.CustomerID == 84
            orderby o.Cost
            select o).Skip(2).Take(2);
```

Element Operators (Single, Last, First, ElementAt, Defaults)

Conversions (ToArray, ToDictionary, ToList, ToLookup)

see https://vslapp.files.wordpress.com/2011/11/linq-cheatsheet.pdf



File I/O

File Handling in .Net

- Mainly implemented in classes of ***System.IO***
- ***Streams*** (with different ***Readers*** and ***Writers***)
- Convenience - Classes
  (***Directory***, ***File***, ***Path***, ***FileSystemWatcher***, . . .)
- Info - Classes
  (***DirectoryInfo***, ***FileInfo***, ***DriveInfo***. . .)